Docker Commands

Complete Reference Guide with Detailed Examples & Explanations

Docker Engine • Docker Compose • Registry • Networking

Basics Images Containers Run Options Exec & Logs Volumes Networks Compose Registry Dockerfile Cleanup Cheat Sheet

Basics & System Info

Version checks and system-wide information

`docker --version`

BASIC

Displays the installed Docker version. Use this to verify Docker is installed.

terminal

```
$ docker --version
Docker version 25.0.3, build 4debf41
```

`docker version`

BASIC

Shows detailed version info for both the Docker client and server (daemon). Useful to verify the client-server connection is working.

terminal

```
$ docker version
Client: Docker Engine
Version: 25.0.3
Server: Docker Engine
Engine: Version: 25.0.3
```

`docker info`

BASIC

Displays system-wide Docker information: number of containers, images, storage driver, CPU, memory, OS, kernel version, and more. Very useful for debugging.

```
# View all system info
```

```
docker info

# Filter specific info using --format
docker info --format '{{.MemTotal}}'
```

Image Commands

Pull, build, tag, inspect and manage Docker images

Searching & Pulling Images

`docker search <image>` BASIC

Search Docker Hub for images. Returns name, description, stars, official flag, and automated flag.

```
# Search for nginx images
docker search nginx

# Search with filter (only official images)
docker search --filter is-official=true nginx

# Show at most 5 results
docker search --limit 5 postgres
```

`docker pull <image>[:tag]` IMPORTANT

Downloads an image from Docker Hub (or a private registry) to your local machine. If no tag is specified, it pulls `latest`.

```
# Pull latest Ubuntu
docker pull ubuntu

# Pull a specific version
docker pull ubuntu:22.04

# Pull from a private registry
docker pull registry.mycompany.com/myapp:1.0

# Pull a specific platform (e.g., ARM64)
docker pull --platform linux/arm64 nginx
```

> 💡 **Tip:** Always pin image versions in production (e.g., `nginx:1.25`) instead of using `latest` to avoid unexpected breaking changes.

Listing & Inspecting Images

`docker images` BASIC

Lists all locally available images, showing Repository, Tag, Image ID, Created date, and Size.

```
# List all images
docker images

# List all including intermediate layers
docker images -a

# Show only image IDs
docker images -q

# Filter by name
docker images nginx

# Custom format output
docker images --format "{{.Repository}}:{{.Tag}} → {{.Size}}"
```

terminal

```
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx latest a72860cb95fd 2 days ago 187MB
ubuntu 22.04 ca2b0f26964c 3 weeks ago 77.9MB
```

docker inspect <image>

ADVANCED

Returns detailed low-level JSON information about an image: config, layers, environment variables, exposed ports, volumes, entrypoint, and more.

```
# Full inspect output (JSON)
docker inspect nginx

# Extract just the exposed ports
docker inspect --format '{{.Config.ExposedPorts}}' nginx

# Get the environment variables
docker inspect --format '{{.Config.Env}}' ubuntu
```

docker history <image>

ADVANCED

Shows the history of an image – each layer, when it was created, what command created it, and the size of each layer. Useful for debugging large image sizes.

```
# Show history of nginx image
docker history nginx

# Show with no truncation
docker history --no-trunc nginx
```

Building Images

docker build

IMPORTANT

Builds a Docker image from a **Dockerfile**. The `.` (dot) is the build context — the directory whose files are sent to the Docker daemon.

```
# Basic build — tag as "myapp"
docker build -t myapp .

# Build with a version tag
docker build -t myapp:2.1 .

# Use a custom Dockerfile name
docker build -f Dockerfile.prod -t myapp:prod .

# Build without using cache (forces fresh build)
docker build --no-cache -t myapp .

# Pass build arguments (available during build)
docker build --build-arg NODE_ENV=production -t myapp .

# Build for multiple platforms (buildx)
docker buildx build --platform linux/amd64,linux/arm64 -t myapp:multi .
```

> 💡 **Tip:** Use `.dockerignore` file (like `.gitignore`) to exclude files from the build context and speed up builds.

Tagging & Removing Images

`docker tag <source> <target>`

BASIC

Creates a new tag for an existing image. Used before pushing to a registry to label the image with the correct repository path and version.

```
# Tag local image for Docker Hub
docker tag myapp johndoe/myapp:latest

# Tag with version number
docker tag myapp:latest johndoe/myapp:2.1.0

# Tag for a private registry
docker tag myapp registry.company.com/team/myapp:v1
```

`docker rmi <image>`

DANGER

Removes one or more images from local storage. You cannot remove an image that is in use by a container (use `-f` to force).

```
# Remove by name
docker rmi ubuntu

# Remove by image ID
docker rmi ca2b0f26964c

# Force remove (even if container uses it)
docker rmi -f nginx
```

```
# Remove all images at once
docker rmi $(docker images -q)
```

> ⚠ **Warning:** Removing images is permanent. Always ensure no running or stopped containers depend on the image first.

`docker image prune`

DANGER

Removes **dangling images** — untagged images that are no longer referenced by any container. A clean and safe way to free up disk space.

```
# Remove dangling (untagged) images
docker image prune

# Remove ALL unused images (not just dangling)
docker image prune -a

# Skip confirmation prompt
docker image prune -a -f
```

`docker save / docker load`

ADVANCED

Save an image to a `.tar` archive file (for transfer or backup), and load it back. Useful for offline environments with no registry access.

```
# Save image to tar file
docker save -o myapp.tar myapp:latest

# Load image from tar file
docker load -i myapp.tar

# Pipe save directly to gzip
docker save myapp:latest | gzip > myapp.tar.gz
```

Container Lifecycle

Create, start, stop, and remove containers

`docker ps`

BASIC

Lists running containers. With `-a`, shows all containers including stopped ones. Displays Container ID, Image, Command, Created, Status, Ports, and Names.

```
# List only running containers
docker ps

# List ALL containers (running + stopped)
docker ps -a
```

```
# Show only container IDs
docker ps -q

# Show last N containers
docker ps -n 5

# Custom format output
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

docker start / stop / restart

IMPORTANT

Control the state of existing containers. `stop` sends a SIGTERM, waits 10s, then SIGKILL. `restart` combines stop + start.

```
# Start a stopped container
docker start mycontainer

# Start and attach to its output
docker start -a mycontainer

# Gracefully stop (waits 10s for cleanup)
docker stop mycontainer

# Stop with custom timeout (30 seconds)
docker stop -t 30 mycontainer

# Stop ALL running containers
docker stop $(docker ps -q)

# Restart a container
docker restart mycontainer
```

docker kill <container>

DANGER

Forcefully terminates a container immediately by sending SIGKILL. Unlike `docker stop`, it does NOT wait for graceful shutdown. Use only when `stop` fails.

```
# Force kill immediately
docker kill mycontainer

# Send a specific signal
docker kill --signal SIGTERM mycontainer
```

docker rm <container>

DANGER

Removes stopped containers and frees their resources. Does NOT remove the image, only the container instance.

```
# Remove a stopped container
docker rm mycontainer

# Force remove a running container
```

```
docker rm -f mycontainer

# Remove multiple containers
docker rm container1 container2 container3

# Remove all stopped containers
docker rm $(docker ps -aq)

# Remove stopped containers (cleaner method)
docker container prune
```

docker rename

BASIC

Renames an existing container. Useful when you have auto-generated names and want something meaningful.

```
# Rename container
docker rename old_name new_name

# Example: rename random name to "webserver"
docker rename vibrant_euler webserver
```

🚀 docker run — All Options

The most powerful command — create and start a container with full control

Basic Run Modes

IMPORTANT

Run containers in foreground, interactive, or detached (background) mode.

```
# Run and exit immediately (no TTY)
docker run ubuntu

# Interactive + TTY (stay in shell)
docker run -it ubuntu bash

# Detached mode (run in background)
docker run -d nginx

# Detached + interactive (for later attach)
docker run -dit ubuntu
```

Naming & Port Mapping

IMPORTANT

Assign a custom name to a container and map host ports to container ports.

```
# Name a container
docker run --name webserver nginx

# Map host port 8080 → container port 80
```

```
docker run -p 8080:80 nginx

# Bind to specific host IP
docker run -p 127.0.0.1:8080:80 nginx

# Map multiple ports
docker run -p 8080:80 -p 8443:443 nginx

# Publish ALL exposed ports to random host ports
docker run -P nginx

# Full example: named, detached, port mapped
docker run -d --name webserver -p 8080:80 nginx
```

Environment Variables

BASIC

Pass environment variables into a container. Essential for configuration like database credentials, API keys, and app settings.

```
# Single environment variable
docker run -e MY_ENV=hello ubuntu

# Multiple environment variables
docker run -e DB_HOST=localhost -e DB_PORT=5432 myapp

# Load from a .env file
docker run --env-file .env myapp

# Example: PostgreSQL with env vars
docker run -d --name postgres-db \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_DB=mydb \
  postgres:15
```

Volume Mounts

IMPORTANT

Mount directories or named volumes into containers to persist data or share files between host and container.

```
# Bind mount: host dir → container dir
docker run -v /home/user/html:/usr/share/nginx/html nginx

# Use current directory
docker run -v $(pwd):/app node:18

# Named volume (Docker manages location)
docker run -v mydata:/app/data myapp

# Read-only mount
docker run -v /config:/etc/config:ro myapp

# Modern --mount syntax (more explicit)
docker run --mount type=bind,source=$(pwd),target=/app node
```

Resource Limits

ADVANCED

Restrict how much CPU and memory a container can use to prevent one container from starving others.

```
# Limit to 512MB RAM
docker run --memory "512m" nginx

# Limit to 1 CPU core
docker run --cpus "1.0" nginx

# Limit CPU and memory together
docker run --memory "256m" --cpus "0.5" myapp

# Set memory + swap (512m RAM, 1GB total)
docker run --memory "512m" --memory-swap "1g" myapp
```

Restart Policies

ADVANCED

Control whether and how Docker automatically restarts containers when they exit or when the Docker daemon restarts.

POLICY	BEHAVIOR
`no`	Never restart (default)
`on-failure`	Restart only if exit code ≠ 0
`always`	Always restart, even after daemon restart
`unless-stopped`	Always restart unless manually stopped

```
# Never restart (default)
docker run --restart no nginx

# Restart on failure (max 3 times)
docker run --restart on-failure:3 myapp

# Always restart (for production services)
docker run -d --restart always nginx

# Unless manually stopped
docker run -d --restart unless-stopped nginx
```

Other Useful Run Options

ADVANCED

```
# Auto-remove container after it exits
docker run --rm ubuntu echo "Hello World"

# Set working directory inside container
docker run -w /app node:18
```

```
# Run as specific user
docker run --user 1000:1000 ubuntu

# Set hostname
docker run --hostname myhost ubuntu

# Add /etc/hosts entry
docker run --add-host db.local:192.168.1.100 myapp

# Override entrypoint
docker run --entrypoint bash myapp

# Real-world full example
docker run -d \
  --name webapp \
  -p 8080:3000 \
  -e NODE_ENV=production \
  -v $(pwd)/logs:/app/logs \
  --memory "512m" \
  --restart unless-stopped \
  myapp:latest
```

Exec, Logs & Monitoring

Debug, monitor, and inspect running containers

`docker exec`

IMPORTANT

Run a command inside an **already-running** container. This is how you "SSH into" a container. Does not start a new container — uses an existing one.

```
# Open interactive bash shell
docker exec -it mycontainer bash

# Use sh (for Alpine-based containers)
docker exec -it mycontainer sh

# Run a single command
docker exec mycontainer ls -la /app

# Run as root user
docker exec -u root -it mycontainer bash

# Run with environment variable
docker exec -e DEBUG=true mycontainer env

# Set working directory
docker exec -w /app -it mycontainer bash

# Detached — run in background
docker exec -d mycontainer touch /app/flag.txt
```

> 💡 **Tip:** `docker exec` requires the container to be running. If it's stopped, use `docker start mycontainer` first.

docker logs

IMPORTANT

Fetch the stdout/stderr logs from a container. Essential for debugging application errors and monitoring behavior.

```
# View all logs
docker logs mycontainer

# Follow/tail logs in real-time
docker logs -f mycontainer

# Show last 50 lines
docker logs --tail 50 mycontainer

# Show logs with timestamps
docker logs -t mycontainer

# Logs since a specific time
docker logs --since 2024-01-01T10:00:00 mycontainer

# Logs in last 30 minutes
docker logs --since 30m mycontainer

# Follow + tail 100 lines + timestamps
docker logs -f --tail 100 -t mycontainer
```

docker stats

ADVANCED

Displays a live stream of resource usage statistics for running containers: CPU %, Memory, Network I/O, Block I/O, and PIDs.

```
# Live stats for all containers
docker stats

# Stats for specific container
docker stats mycontainer

# One-shot (no live stream)
docker stats --no-stream

# Custom format
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

docker top / docker port / docker diff

ADVANCED

Useful debugging utilities to inspect processes, port mappings, and filesystem changes in containers.

```
# List processes inside container
docker top mycontainer

# Show port mappings for a container
```

```
docker port mycontainer

# Show filesystem changes (A=added, C=changed, D=deleted)
docker diff mycontainer

# Attach to a running container's stdout
docker attach mycontainer
```

docker cp

BASIC

Copy files or directories between the host and a container. Works on both running and stopped containers.

```
# Copy FILE from container to host
docker cp mycontainer:/app/logs/error.log ./error.log

# Copy FOLDER from container to host
docker cp mycontainer:/app/data ./backup/

# Copy file FROM host TO container
docker cp ./config.json mycontainer:/app/config.json

# Copy folder FROM host TO container
docker cp ./src/ mycontainer:/app/src/
```

Volumes — Persistent Storage

Persist data beyond container lifecycles

Volume Management

IMPORTANT

Docker volumes are the preferred way to persist data. They are managed by Docker and stored in a Docker-managed directory on the host.

```
# Create a volume
docker volume create myvolume

# List all volumes
docker volume ls

# Inspect a volume (see mount point)
docker volume inspect myvolume

# Remove a volume
docker volume rm myvolume

# Remove all unused volumes
docker volume prune

# Use volume in a container
docker run -v myvolume:/app/data myapp

# Use --mount syntax (more readable)
```

```
docker run --mount source=myvolume,target=/app/data myapp

# Bind mount: use host directory directly
docker run -v /home/user/data:/app/data myapp
```

TYPE	SYNTAX	BEST FOR
Named Volume	`-v myvolume:/data`	Production data persistence
Bind Mount	`-v /host/path:/data`	Development (live code reload)
tmpfs	`--tmpfs /tmp`	Sensitive/temp data in RAM

Docker Networking

Connect containers and manage network isolation

Network Management

IMPORTANT

Docker networks allow containers to communicate with each other securely. By default, containers on the same network can reach each other by name.

```
# Create a bridge network
docker network create mynetwork

# Create with specific driver and subnet
docker network create --driver bridge --subnet 192.168.10.0/24 mynetwork

# List all networks
docker network ls

# Inspect a network
docker network inspect mynetwork

# Run container on a network
docker run --network mynetwork --name app myapp

# Connect existing container to network
docker network connect mynetwork mycontainer

# Disconnect from network
docker network disconnect mynetwork mycontainer

# Remove a network
docker network rm mynetwork

# Remove all unused networks
docker network prune
```

NETWORK DRIVER	DESCRIPTION
`bridge`	Default. Isolated network for containers on the same host
`host`	Container shares host's network stack (no isolation)

`none`	No networking — fully isolated
`overlay`	Multi-host networking (Docker Swarm)

🗂 Docker Compose

Define and run multi-container apps with a YAML file

Core Compose Commands

IMPORTANT

```
# Start all services (foreground)
docker compose up

# Start detached (background)
docker compose up -d

# Rebuild images before starting
docker compose up --build

# Start specific service only
docker compose up -d web

# Stop + remove containers, networks
docker compose down

# Also remove volumes
docker compose down -v

# Stop without removing
docker compose stop

# Start stopped services
docker compose start

# Restart all services
docker compose restart

# List running services
docker compose ps

# View logs for all services
docker compose logs

# Follow logs for specific service
docker compose logs -f web

# Run a command in a service
docker compose exec web bash

# Run a one-off command (creates new container)
docker compose run --rm web python manage.py migrate

# Scale service to N replicas
docker compose up --scale web=3 -d
```

```
# Pull latest images
docker compose pull

# Build all service images
docker compose build

# Use custom compose file
docker compose -f docker-compose.prod.yml up -d
```

docker-compose.yml — Full Example

IMPORTANT

A real-world example with web app, database, Redis cache, and proper networking and volumes.

```yaml
version: "3.9"

services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
      - REDIS_HOST=cache
    volumes:
      - ./src:/app/src
    depends_on:
      - db
      - cache
    networks:
      - appnet
    restart: unless-stopped

  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - appnet

  cache:
    image: redis:7-alpine
    networks:
      - appnet

volumes:
  pgdata:

networks:
  appnet:
    driver: bridge
```

Registry & Sharing Images

Login, push and pull from Docker Hub or private registries

Login, Push & Pull

IMPORTANT

```
# Login to Docker Hub
docker login

# Login with credentials inline
docker login -u myuser -p mypassword

# Login to private registry
docker login registry.mycompany.com

# Logout
docker logout

# Tag image for Docker Hub
docker tag myapp johndoe/myapp:1.0

# Push to Docker Hub
docker push johndoe/myapp:1.0

# Push to private registry
docker tag myapp registry.mycompany.com/myapp:1.0
docker push registry.mycompany.com/myapp:1.0

# Pull from private registry
docker pull registry.mycompany.com/myapp:1.0
```

Dockerfile Instructions

All Dockerfile instructions explained with examples

All Dockerfile Instructions

IMPORTANT

INSTRUCTION	DESCRIPTION
`FROM`	Base image to start from
`RUN`	Execute command during build (creates a new layer)
`COPY`	Copy files from host to image
`ADD`	Like COPY but supports URLs and tar extraction
`CMD`	Default command when container starts (overridable)
`ENTRYPOINT`	Fixed command; CMD arguments are appended to it

`ENV`	Set environment variable (persists in container)
`ARG`	Build-time variable (NOT persisted in container)
`EXPOSE`	Document which port the container listens on
`WORKDIR`	Set working directory for subsequent instructions
`VOLUME`	Create a mount point
`USER`	Set user for subsequent instructions
`LABEL`	Add metadata as key=value pairs
`HEALTHCHECK`	Define health check command
`ONBUILD`	Trigger instruction when used as base image

Production-Ready Node.js Dockerfile

ADVANCED

```
# Stage 1: Build
FROM node:18-alpine AS builder

WORKDIR /app

# Copy dependency files first (cache optimization)
COPY package*.json ./
RUN npm ci --only=production

COPY . .
RUN npm run build

# Stage 2: Production image (smaller)
FROM node:18-alpine

LABEL maintainer="[email protected]"
LABEL version="1.0"

WORKDIR /app

# Copy only production artifacts
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

ENV NODE_ENV=production
ENV PORT=3000

EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget --quiet --tries=1 http://localhost:3000/health || exit 1

# Run as non-root user (security)
USER node

CMD ["node", "dist/server.js"]
```

💡 **Multi-stage builds** drastically reduce final image size by only copying what's needed into the final stage, leaving behind build tools, dev dependencies, and source files.

Cleanup & Disk Management

Free up disk space by removing unused Docker resources

System Cleanup Commands

DANGER

```
# Show Docker disk usage
docker system df

# Detailed breakdown
docker system df -v

# Remove stopped containers + dangling images + unused networks
docker system prune

# Also remove unused images (not just dangling)
docker system prune -a

# ALSO remove volumes (⚠ DATA LOSS)
docker system prune -a --volumes

# Skip confirmation
docker system prune -a -f

# Remove only containers
docker container prune

# Remove only images
docker image prune -a

# Remove only volumes
docker volume prune

# Remove only networks
docker network prune
```

⚠ **Warning:** `docker system prune -a --volumes` will delete ALL unused resources including volume data. Make sure to backup important data first!

Quick Cheat Sheet

Most-used commands at a glance

🖼 Images

`docker pull nginx`	Download image

`docker images`	List images
`docker build -t app .`	Build image
`docker rmi nginx`	Delete image
`docker image prune -a`	Clean unused

📦 Containers

`docker ps -a`	List all containers
`docker start web`	Start container
`docker stop web`	Stop container
`docker rm web`	Remove container
`docker container prune`	Clean stopped

🚀 Run

`docker run -it ubuntu bash`	Interactive
`docker run -d nginx`	Background
`docker run -p 8080:80 nginx`	Port map
`docker run --rm ubuntu cmd`	Auto-cleanup
`docker run -v vol:/data app`	Volume

🔍 Debug

`docker logs -f web`	Follow logs
`docker exec -it web bash`	Shell access
`docker stats`	Live stats
`docker inspect web`	Full details
`docker top web`	Processes

🗂 Compose

`docker compose up -d`	Start services
`docker compose down`	Stop services
`docker compose logs -f`	Follow logs
`docker compose ps`	List services
`docker compose exec web bash`	Shell

🧹 Cleanup

`docker system df`	Disk usage
`docker system prune`	Clean up
`docker system prune -a`	Full clean
`docker volume prune`	Clean volumes
`docker network prune`	Clean networks